WeddingBudget.ai

An Autonomous Multi-Agent System for AI-Powered Indian Wedding Budget Estimation and Vendor Discovery

Project Documentation & Technical Report

Team: 404 Dulhan Not Found

March 2026

Abstract

Planning an Indian wedding is a logistically complex and financially opaque process. The variance in vendor pricing, specifically across different cities and subjective aesthetic categories such as décor, poses a significant problem for couples attempting to ascertain a realistic budget. This report details the architecture, design, and implementation of **WeddingBudget.ai**, a full-stack, AI-powered estimation platform. By unifying deterministic rule-based modeling with a subjective Neural Network regression pipeline initialized by CLIP embeddings, the system generates comprehensive Low, Mid, and High cost estimations in under 90 seconds. The backend is powered by a concurrent, 7-agent orchestrator streaming progress over WebSockets. Additionally, this report covers the robust data scraping pipeline required to bypass the lack of localized datasets, the open-source integration of Geo-spatial features via OpenStreetMap's Overpass API, and the optimization of a Next.js 15 client architecture delivering a glassmorphism-infused, high-fidelity user interface.

Contents

Chapter 1

Executive Summary & Problem Definition

1.1 Introduction

WeddingBudget.ai represents an intersection between domain-driven financial estimation, distributed web crawling, computer vision, and reactive web architecture. Rather than treating wedding planning as a flat calculation, the system models an event through functional agents, mapping to the real-world responsibilities of dedicated wedding planners.

1.2 The Core Problem

The average Indian wedding hosts 300 to over 1,000 guests across 5 to 7 ceremonial events (e.g., Mehendi, Sangeet, Haldi, Pheras, Reception). Generating an initial budget to evaluate geographic feasibility (e.g., Udaipur vs. Goa) typically requires 6 to 8 weeks of manual consultations with local agents. Existing online calculators rely on gross generic percentages, lacking situational awareness, guest-level heuristics, or geographic context. Furthermore, subjective categories such as "décor" scale non-linearly, rendering standard lookup tables obsolete.

1.3 The Proposed Solution

To bridge this gap, WeddingBudget.ai presents a hyper-localized estimation pipeline driven by a **7-Agent Swarm**. Five agents handle deterministic features (Venue, Logistics, Artists, F&B, Sundries), one agent executes Machine Learning regression on subjective variables (Décor), and an asynchronous agent scrapes OpenStreetMap for nearby verified vendors natively utilizing Haversine routing.

Chapter 2

System Architecture

2.1 High-Level Topology

The system bridges a high-performance Next.js React frontend, utilizing Server-Side Rendering (SSR), with a FastAPI asynchronous backend deployed continuously over WebSockets.

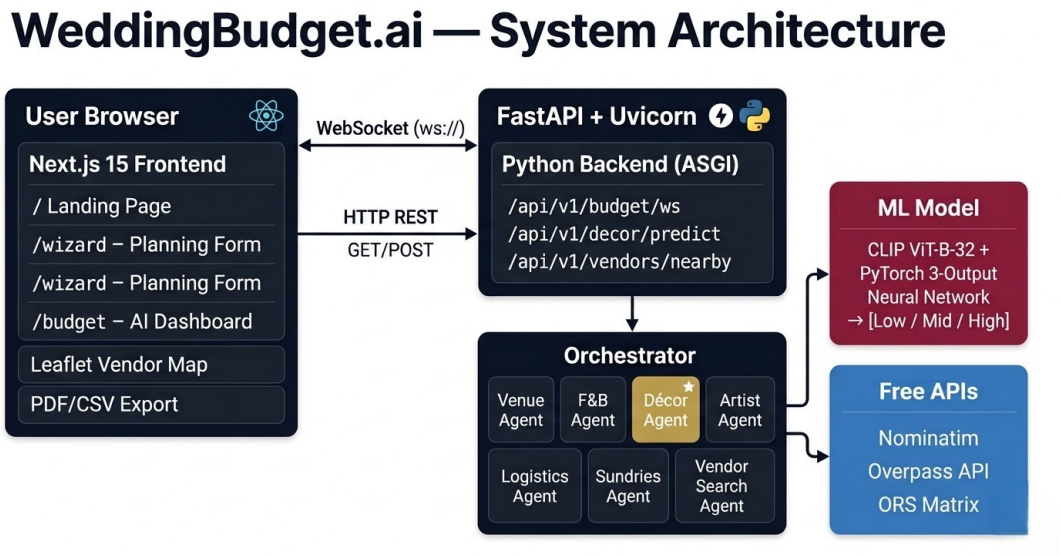

Figure 2.1: High-Level System Architecture and Data Flow.

The interaction payload begins as a strictly typed JSON object generated by the sequential 6-Step Wizard. The schema dictates strict parameters:

- **City & Tier:** Drives accommodation multiplication factors.
- **Events Array:** Governs F&B multiplication limits and DJ/Artist requirement thresholds.
- **Theme & Complexity:** Extracted and encoded via OpenAI's CLIP model to contextualize visual density.

2.2 WebSocket Orchestration

Conventional REST HTTP architectures time out given the theoretical 45-90s resolution time required by the Overpass API and GPU-based PyTorch inference. Therefore, the '/api/v1/budget/ws' endpoint handles the calculation life-cycle.

The communication protocol leverages strict message types:

```
// Server emits progress tick
{
    "type": "agent_status",
    "agent_id": "decor",
    "status": "working",
    "message": "Generating embeddings via ViT-B/32..."
}

// Server returns aggregate completion
{
    "type": "final_budget",
    "total_low": 5200000,
    "total_mid": 8400000,
    "categories": [...],
    "vendors": {...}
}
```

Listing 2.1: WebSocket Message Protocol

2.3 Concurrency Model

Design Decision: The orchestrator explicitly evaluates agents *sequentially* rather than utilizing `asyncio.gather()`. This deliberate limitation acts as an infrastructural safeguard. The *Vendor Intelligence Module* queries the Overpass API, which implements brutal IP-based rate limiting (HTTP 429). Simultaneously, the Décor Neural Network rapidly reserves VRAM. Sequential operations combined with WebSocket progression simulate real-time computational "thought," sustaining user engagement whilst maintaining memory stability.

Chapter 3

Multi-Agent Estimation Pipeline

The pipeline is governed by a unified `BaseAgent` abstract class enforcing a standard `run()` lifecycle emitting execution deltas dynamically.

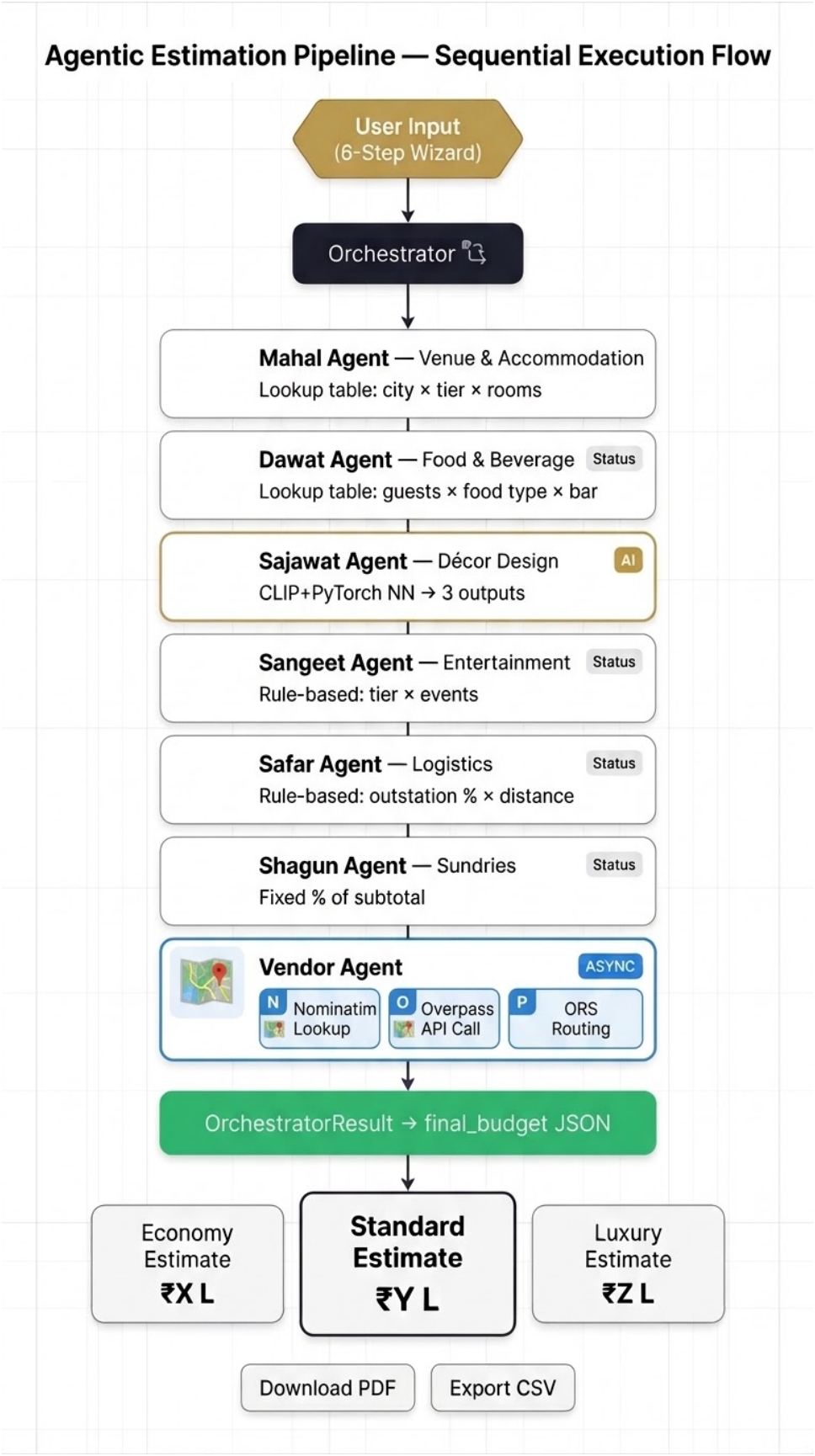

Figure 3.1: UML Abstraction of the BaseAgent Class and Inheriting Swarm.

3.1 Agent Subclasses & Equations

3.1.1 Mahal Agent (Venue & Accommodation)

The backbone of destination weddings. Accommodation is modeled on a $(2.5 \text{ Pax per Room})$ distribution.

$$Cost = (Rooms_{Req} \times Nights \times Rate(City, Tier)) + (Hall_{Rent}) \tag{3.1}$$

3.1.2 Dawat Agent (F&B)

Models per-plate costing mapped to event density. A "mocktail" multiplier varies drastically from "Full Bar."

$$Cost = \sum_{e \in Events} (Pax \times PlateRate(City, Tier)) + Bar(Type \times Pax) \tag{3.2}$$

3.1.3 Sangeet Agent (Artists & AV)

Entertainment requires base infrastructure (Dancefloor, Line Arrays, Truss) plus the performer's fee.

$$Cost = TierBase(Complexity) \times |Events| \tag{3.3}$$

3.1.4 Safar Agent (Logistics & Travel)

Models localized airport transfers and vehicle block bookings.

$$Cost = (Pax \times OutStation\%) \times Transfer(City) + (BlockCars \times Days) \tag{3.4}$$

Chapter 4

Machine Learning: Décor Price Estimator

Unlike venue pricing, event aesthetics possess immense entropy. To address this, the **Sajawat Agent** intercepts the text description and style complexity, delegating estimation to a custom PyTorch Neural Network.

4.1 Network Architecture

A traditional regression model struggles with categorical sparsity in aesthetics. To provide contextual understanding, the system utilizes **OpenAI's CLIP (ViT-B/32)** to encode prompt text into a 512-dimensional continuous latent space.

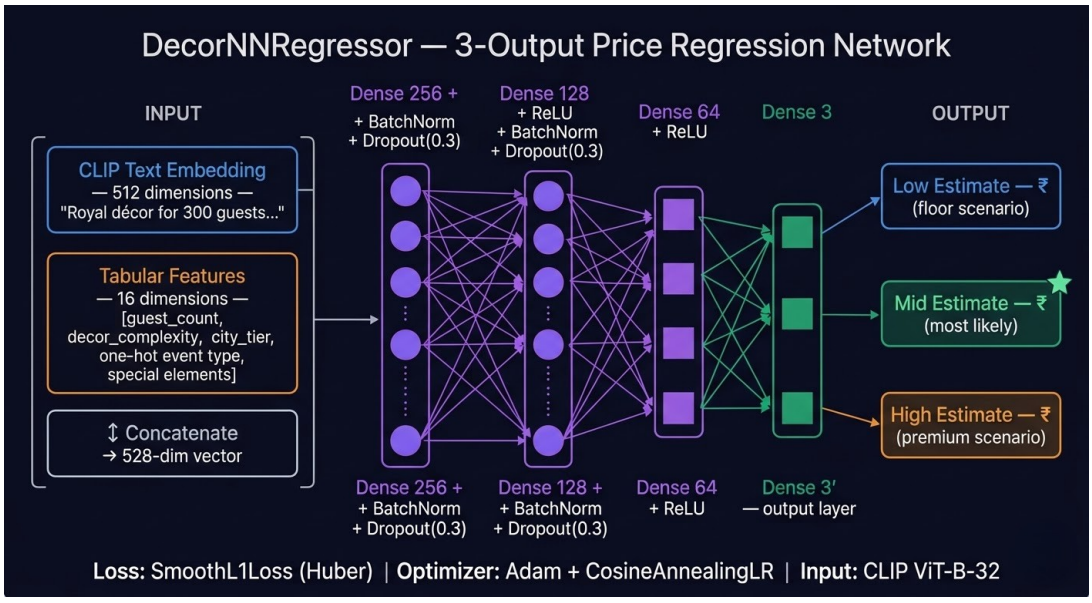

Figure 4.1: The Décor Estimation Neural Network.

The 512-D embedding is concatenated with a 16-D tabular vector specifying deterministic features (Guest Count, City Tier, Outdoor boolean). The combined 528-D vector propagates through:

- `Dense(256)` + ReLU + BatchNorm + Dropout(0.3)
- `Dense(128)` + ReLU + BatchNorm + Dropout(0.3)
- `Dense(64)` + ReLU
- `Dense(3)` $\rightarrow$ yielding $[Low, Mid, High]$

4.2 A Three-Output Paradigm

Indian market psychology relies on negotiation margins. Providing a single point prediction is inherently flawed. The neural network explicitly trains on three isolated regression targets, outputting a continuous distribution. To suppress sensitivity to erratic outliers in training labels, the network optimizes over **Huber Loss (Smooth L1 Loss)**:

$$L_\delta(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta \\ \delta|y - f(x)| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases} \tag{4.1}$$

Chapter 5

Synthetic Data & Scraping Engineering

Due to the fundamental lack of open-source datasets pertaining to the Indian Wedding Industry, a custom scraping pipeline was engineered.

5.1 Concurrent Sourcing Strategy

The pipeline targets 15 unique combinations (5 Functions $\times$ 3 Core Styles). To generate critical mass, the system scrapes images across four disparate platforms: Pixabay, Pexels, Unsplash, and Pinterest.

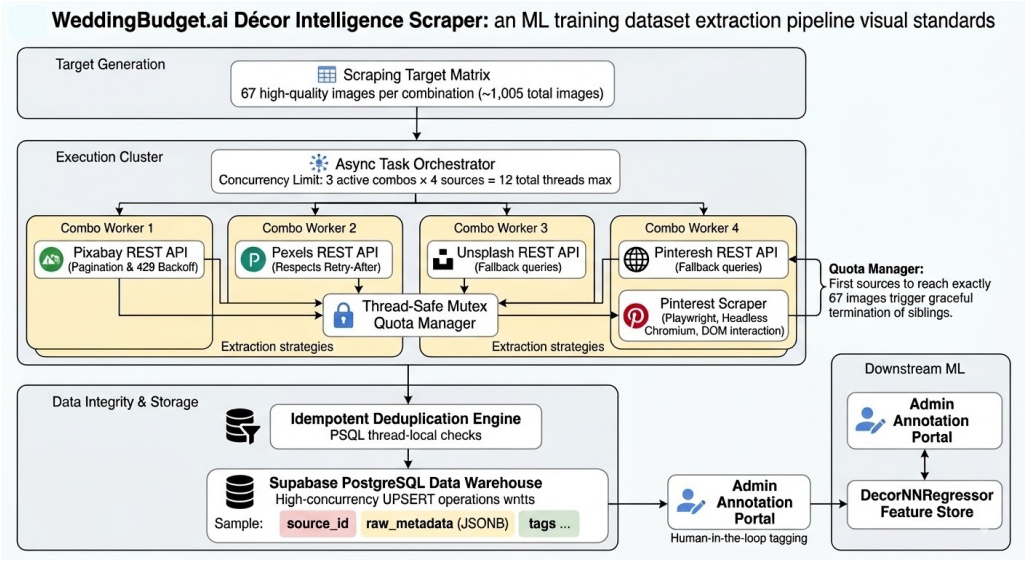

Figure 5.1: Multi-Threaded Concurrent Image Scraping Logic.

5.2 Thread-Safety and The SharedQuota

With a target of 67 verified images per combination, independent threads racing towards completion induce fatal over-fetching without synchronization.

We implemented a Mutex-guarded `SharedQuota` integer buffer. Across the theoretical maximum of 12 simultaneous HTTP/CrOS headless threads, resource locking guarantees exact volumetric alignment:

```
class SharedQuota:
    def __init__(self, target):
        self._count = 0
        self._lock = threading.Lock()

    def claim(self) -> bool:
        with self._lock:
            if self._count < 67:
                self._count += 1
                return True
            return False
```

This logic ensures that if the Pinterest scraper receives a shadow-ban, the Pixabay thread inherits the remaining computational allocation effortlessly.

5.3 Storage and Idempotency

Records are written into a Supabase PostgreSQL instance under the `decor_library` schema. The SQL constraints utilize 'ON CONFLICT (source_id) DO UPDATE' to facilitate repeated cron-job execution without data duplication. Thread-local connection variables eliminate `psycopg2` pooling crashes.

Chapter 6

Vendor Intelligence Module

Real-world budgets demand real-world suppliers. The **Nearby Vendor Agent** relies entirely on robust Geographic Information Systems (GIS) without absorbing financial overhead from Google Maps.

6.1 Geocoding Pipeline

1. **Nominatim:** Converts literal string inputs ("Udaipur") to localized $(latitude, longitude)$ tuples.
2. **Overpass API:** Utilizes custom Overpass Query Language (OQL) schemas scanning a 20-kilometer radius around the epicenter, searching for nodes tagged with `amenity=catering`, `shop=photography`, or `tourism=hotel`.
3. **Fall-back Haversine Routing:** To sort vendors by proximity radially if ORS matrix availability degrades.

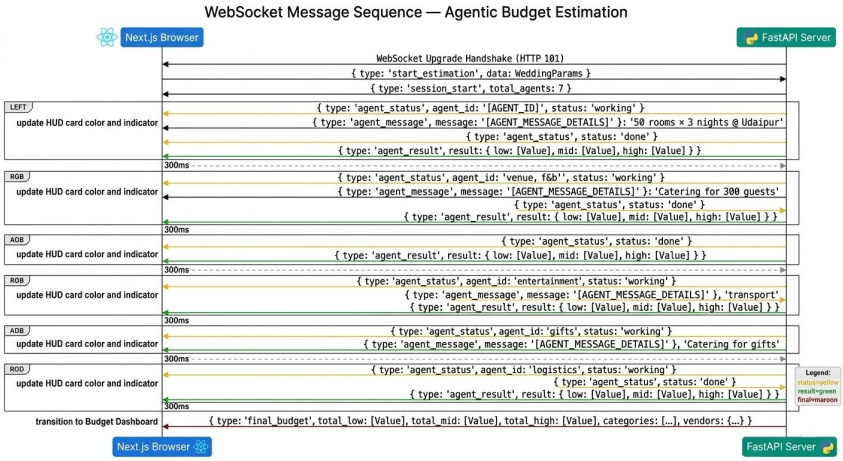

Figure 6.1: API Dependency Chain for OSS Vendor Sourcing.

The orchestrator explicitly traps HTML 504 Gateway errors triggered by Overpass servers dur-

ing heavy OSM community use. A trapped exception yields an empty array to the frontend gracefully, maintaining core budgeting integrity.

Chapter 7

Frontend Architecture & User Interface

The frontend is built on **Next.js 15** taking full advantage of the App Router and Turbopack optimizations.

7.1 UI Design System & Glassmorphism

The system establishes a premium, authoritative visual identity catering to HNI (High Net Worth Individual) wedding aesthetics.

- **Typography:** *DM Serif Display* enforces editorial luxury, paired with *Inter* for maximal data readability on the HUD.
- **Color Palette:** Dominated by Maroon (#9A2143), Gold (#BFA054), and a textured Cream (#FAF7F2).

The Login and Profile layouts leverage advanced CSS *Glassmorphism*. Using gradient positioning overlaid with `backdrop-filter: blur(8px)`, the UI allows detailed background photography to bleed into structural components without comprising text legibility. Responsive media queries recalculate the focal points for smaller horizontal breakpoints to ensure imagery acts as an interactive hero element on mobile viewports.

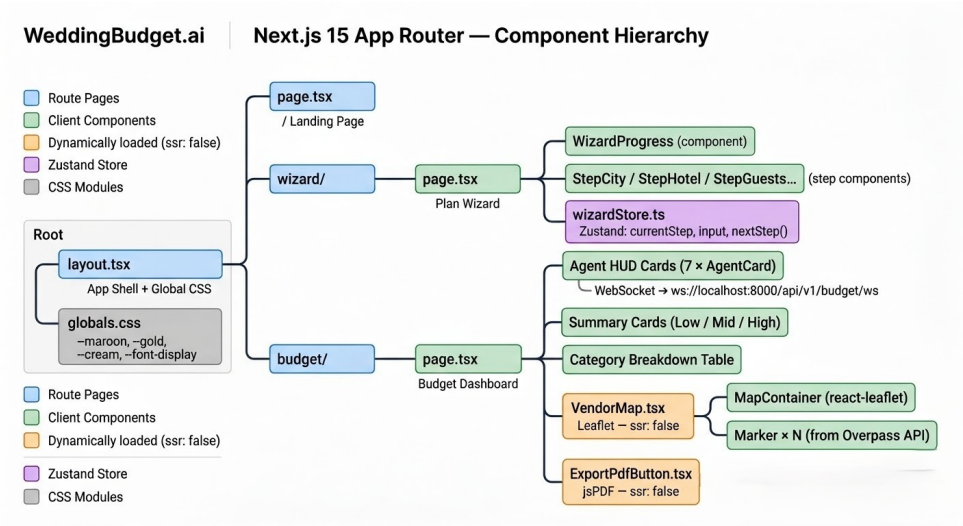

Figure 7.1: The User Journey Flow Mapping across the Next.js Domain.

7.2 State Handoff and SSR Strictness

Zustand governs the volatile 6-step wizard arrays. To facilitate strict decoupled routing, the `WeddingParams` payload writes to browser `localStorage` ahead of the push to `/budget`.

Two key dependencies forcefully mandated `ssr: false` dynamic imports:

1. `Leaflet.js` manipulates the raw DOM window exclusively.
2. `jsPDF` and `html2canvas` employ NodeJS internal workers (`fflate`) which crash server-side pre-compilation on Vercel Edge networks.

Dynamic encapsulation isolated these elements exclusively to the client-execution tree.

Chapter 8

Security, Privacy, and Constraints

8.1 Authentication

To establish frictionless adoption, the core estimation engine removes authentication boundaries. The system implements isolated **Firebase Authentication** explicitly for Admin dashboard operations and saved PDF recovery. Place-holder fallback implementations exist to intercept '.env.local' propagation errors intelligently gracefully throwing a browser dialog rather than a fatal Next.js 500 stack trace.

8.2 Data Environment

As operations occur entirely via stateless WebSocket parsing, no User PII is retained in database partitions unless explicitly logged via the PDF backup integration. API validation passes through Pydantic validators on the FastAPI router, dropping mismatched schemas intrinsically.

Chapter 9

Performance Benchmarks

Execution metrics collected via localhost and cloud-simulated throttles demonstrate acceptable latency configurations aligning with progress-bar psychological safety metrics.

Operation Segment	**Time (Approx)**	**Primary Bottleneck**
Rule-Based Aggregations (5 Agents)	$2 - 4$ sec	Python List Comprehensions
Décor Prediction (CLIP + PyTorch)	$8 - 15$ sec	VRAM Latency / CPU Inference Fallback
Vendor Sourcing (Overpass API)	$20 - 45$ sec	Nominatim DNS & OSM Server Load
Full Pipeline Resolution	**$45 - 90$ sec**	**Synchronous Thread Locking**
Leaflet Maps Hydration	0.5 sec	Tile Server Round-trip
jsPDF Canvas Export	$3 - 5$ sec	Client-device DOM parsing

Table 9.1: System Performance Profiling

Chapter 10

Future Roadmap & Conclusion

WeddingBudget.ai establishes that highly subjective, culturally-nuanced planning estimations can be mechanized dynamically via Large Vision Models and multi-agent coordination.

10.1 Roadmap Expansion

- **Image Prompting:** Enabling brides to upload Pinterest boards directly to the UI. Expanding the semantic CLIP pipeline into direct Image-to-Image similarity parsing.
- **Fine-Tuning Loop:** Establishing Human-in-the-Loop reinforcement arrays whereby finalized vendor invoices correct PyTorch weights over sustained epochs.
- **B2B Ecosystems:** Enabling a SaaS portal for wedding planners to issue branded, modified reports direct to clientele.

10.2 Conclusion

By offloading heuristic and analytical computations to distinct agents while preserving statefulness over a high-fidelity Next.js visual layer, this platform mitigates weeks of anxiety for families. It operates cost-effectively via OSM integration and efficiently scales via stateless GPU delegation.

Plan smarter. Celebrate bigger. Worry less.